

Shooting Game Features and Implementation Logic

1. BPC_04_VRShoot (VR Character Shooting Control Component)

1.1 Overview

Purpose

BPC_04_VRShoot is a newly written VR character Blueprint component supporting the shooting game's input control. Shooting and throwing weapons trigger completely differently: throwing weapons fire on release, while a gun fires from the trigger while held. Attached to the VR character, this component records the pressed state of both hands' trigger and grip buttons and switches the input mapping context based on whether a gun is held, resolving the trigger's conflict between shooting and UI interaction.

Description

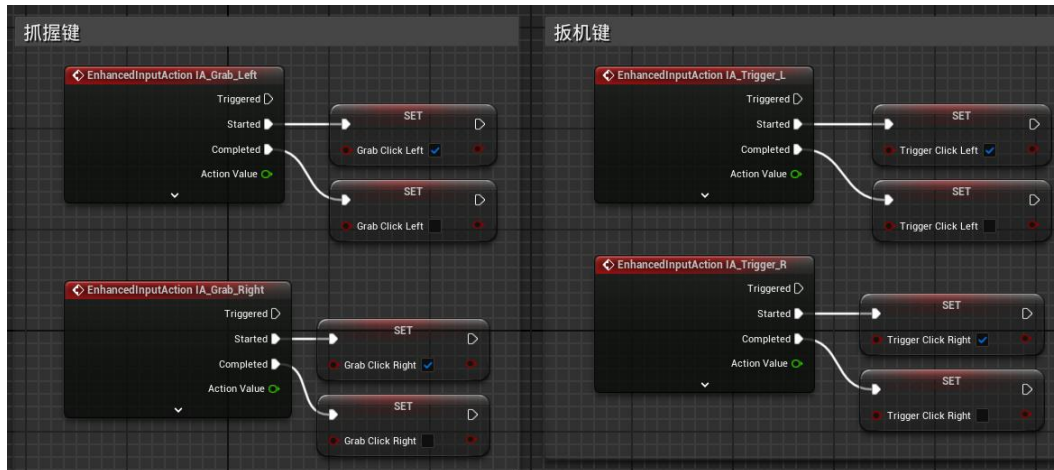
- Component initialization: caches the VR character's left and right hand references in BeginPlay
- Key state recording: converts the pressed/released events of four input actions (both hands' grip and trigger) into boolean state variables
- Input mapping switch: provides the ToggleMappingContext event, called by the gun on pickup/drop to switch between the shooting and UI input mappings
- Combined with the IMC priority adjustment, it fully resolves the trigger's multi-purpose conflict

1.2 Key State Recording

Purpose

Listens to both hands' grip and trigger input actions, converting instantaneous key events into boolean state variables that can be continuously polled by the gun's fire check.

Blueprint screenshots



Notes

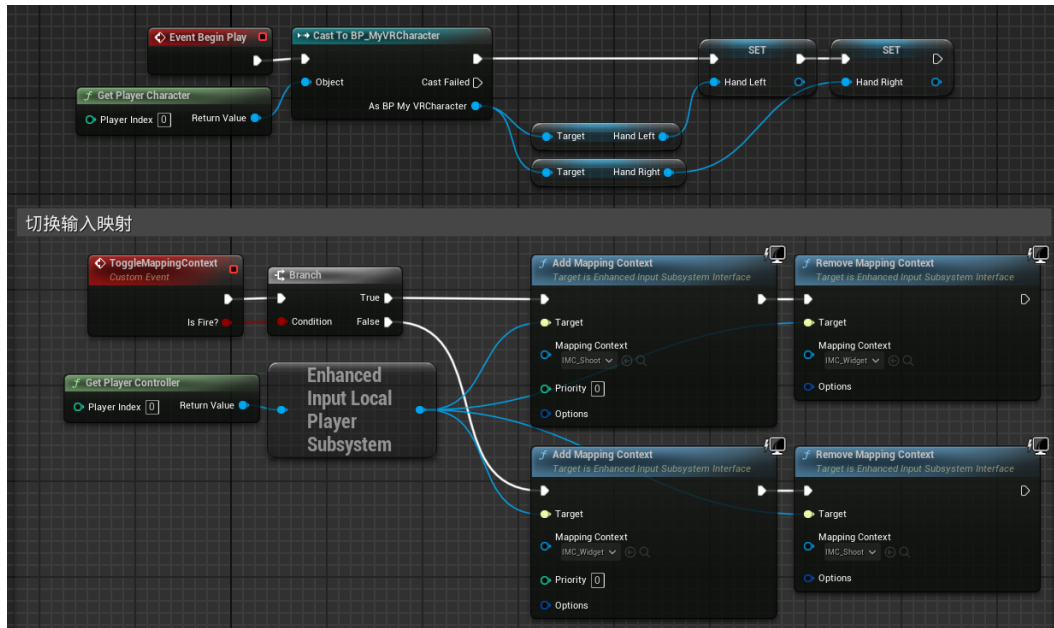
- Started/Completed each fire only once at the press and release instants, not repeatedly while held, so the variable state is stable and suits Tick polling
- The Triggered pin is unused: Triggered fires every frame while held, suiting full-auto fire; this game is semi-automatic and does not need it

1.3 Component Initialization and Input Mapping Switch

Purpose

Caches the VR character's left and right hand references in BeginPlay; provides the ToggleMappingContext custom event, which switches the input mapping context when the gun is picked up/dropped, resolving the conflict of the trigger being used for both shooting and UI interaction.

Blueprint screenshots



Key points

- Conflict background: the VR character's UI interaction (Widget Interaction) uses the trigger, and shooting uses it too, so both fire simultaneously
- Solution: disable UI interaction input while holding the gun and restore it when the gun is put down. You don't need to operate the UI while shooting, so the logic is sensibly mutually exclusive

1.4 Input Mapping Priority Adjustment

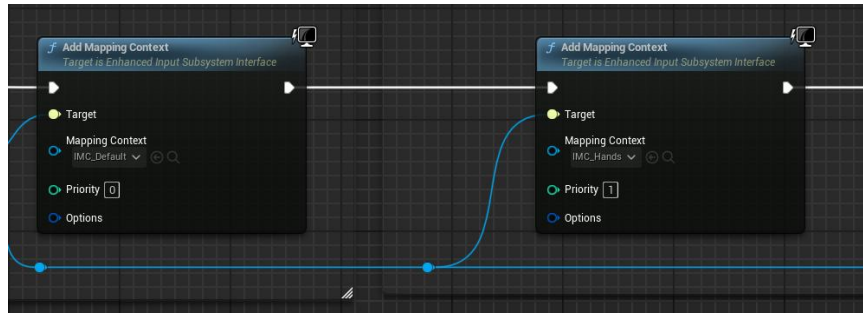
Purpose

Adjusts the input mapping contexts' priority to resolve the key conflict of the trigger driving both hand animation and shooting input.

Key points

- IMC_Default (default input mapping) priority is set to 0
- IMC_Hands (the input mapping controlling hand animation) priority is raised to 1
- In the Enhanced Input system a larger priority value responds first; when the trigger is used by both hand animation and shooting, the hand animation mapping gets priority

Blueprint screenshots



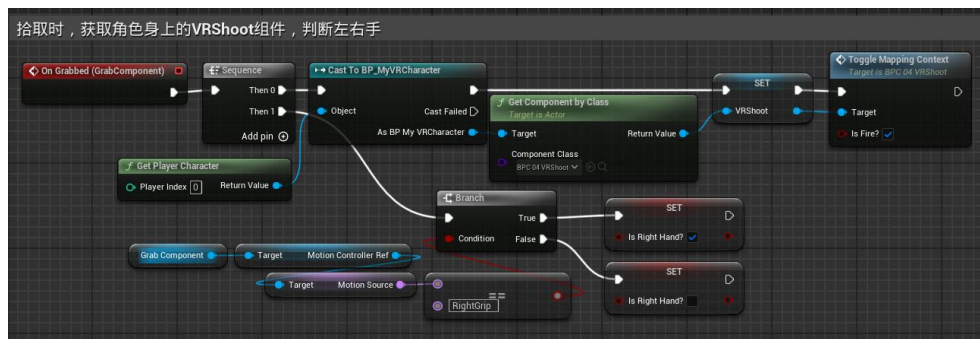
2. Gun (BP_Weapon_Base Shooting Subclass)

2.1 Getting the VRShoot Component and Determining the Hand on Pickup

Purpose

When the player grabs the gun, it caches the BPC_04_VRShoot component reference on the VR character, determines whether the left or right hand holds it, and switches the input mapping to enable shooting input.

Blueprint screenshots



Key points

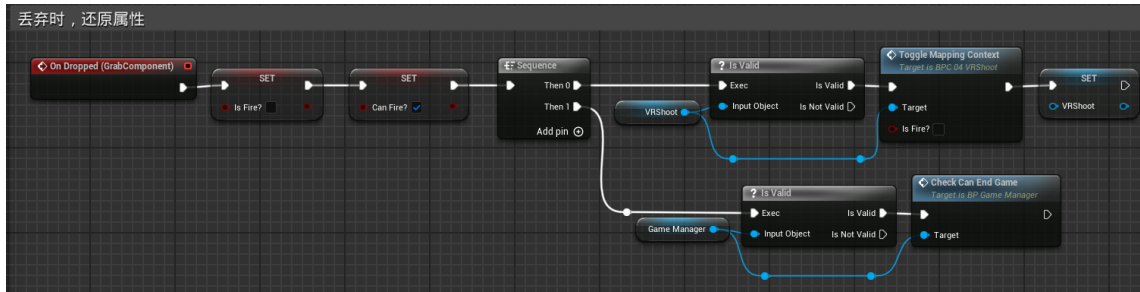
- Is Right Hand? decides which hand's trigger state variable the later Tick fire check reads (Trigger Click Right or Trigger Click Left)
- The VRShoot component reference is the precondition for the gun polling key states; when not held the reference is invalid, so the fire check naturally fails

2.2 Restoring Properties on Drop

Purpose

When the player puts the gun down, it restores the fire state variables, switches back to the default input mapping, clears the component reference, and checks whether the game end condition is met.

Blueprint screenshots



Key points

- Restoring the state variables ensures the fire logic starts fresh when the gun is picked up again, with no leftover shooting state
- After dropping the gun the VRShoot reference is cleared, Tick's Is Valid check fails, and the fire logic automatically goes inert

2.3 Fire Event: Line Trace and Scoring

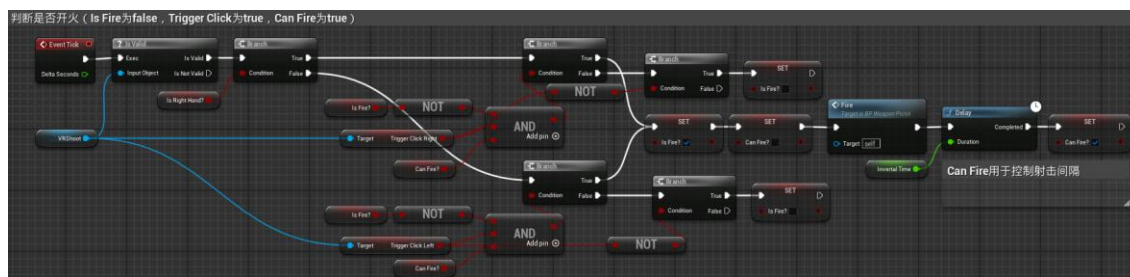
Purpose

On firing, a ray is cast from the muzzle along the forward direction to detect a shooting target; on a hit the impact point is passed to the target for scoring. It also checks and consumes the GameManager's remaining attempts. Because shooting uses a line trace instead of launching a projectile, a new Fire event was written and the original Shoot event (built for throwing weapons) is no longer used.

Execution flow

- Then 0: line trace and hit notification
- Then 1: attempt check and consumption
- Then 2: play sound, animation, and effects

Blueprint screenshots



Key points

- Is Fire? (anti auto-fire): set true at the instant of firing, blocking Tick's repeated triggering while the trigger is held; restored to false when the trigger is released or the weapon dropped. The trigger must be released and pressed again for the next shot
- Trigger Click Left/Right (trigger state): key state variables recorded by BPC_04_VRShoot; with Is Right Hand?, the corresponding hand is read
- Can Fire? (fire interval): defaults to true; set false on firing and restored true after Delay(Interval Time); also set back to true on dropping the weapon. It is independent of Is Fire? and only controls the fire interval — the key can be quickly pressed several times within a second, so "release and press again" alone cannot limit the fire rate; Can Fire? forces at least Interval Time between two shots
- The three variables' duties do not overlap: Is Fire? handles "one shot per press", Can Fire? handles "minimum interval between shots", Trigger Click handles "which hand pressed the trigger"

State table

Is Fire?	Trigger Click	Can Fire?	Result	State change
false	true	true	Fire (call Fire)	Is Fire? = true, Can Fire? = false
false	true	false	No fire (interval cooling)	State unchanged
false	false	Any	No fire (trigger not pressed)	State unchanged
true	true	Any	No fire (already fired for this press)	State unchanged
true	false	Any	No fire, and restore Is Fire? = false	Waiting for the next press

Notes

- The Event Tick boolean polling design converts "input events" into "state queries", unifying left- and right-hand logic in one condition chain
- Interval Time is an adjustable interval variable; larger slows the fire rate, smaller speeds it up

2.5 Fire Presentation (Sound + Slide Recoil + Muzzle Flash)

Purpose

Provides complete audiovisual feedback on firing: the fire sound, slide recoil animation, and muzzle flash effect, all triggered in the same frame as the line trace.

Key points

- The gun model consists of two meshes, the body and the slide: SM_Pistol_Slide is a separate slide mesh component achieving recoil through procedural displacement, with no skeletal animation assets needed

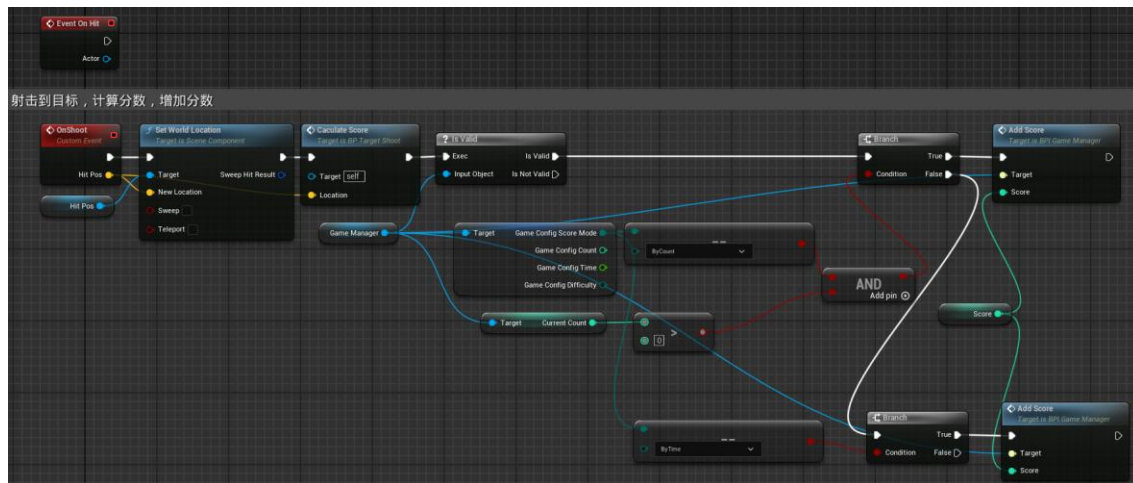
3. Shooting Target (BP_Target_Shoot)

3.1 Hit Scoring Flow (On Shoot)

Purpose

Receives the impact point from the gun's Fire event, moves the bullet hole to the hit position, computes the score for that point, and adds it to the GameManager per the current scoring mode.

Blueprint screenshots



Key points

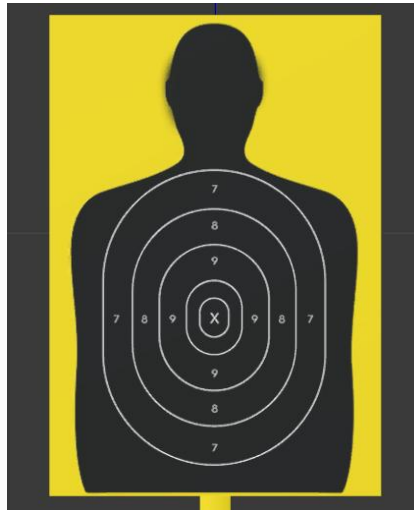
- The ByCount path carries an attempt condition (involving Current Count), forming double insurance with the gun-side pre-check: hits during the 2-second game-end delay do not score
- The ByTime path needs no attempt check: when the countdown hits zero the game ends immediately, so there is no delay window

3.2 Target Face Structure and Score Zones

Purpose

The shooting target uses a human silhouette whose scoring area is concentric elliptical rings on the chest, scoring decreases from inside out: center X ring 10, inner ring 9, middle ring 8, outer ring 7, off-target 0.

Target face reference



Difference from dartboard scoring

- Dartboard: needs two dimensions — angle (20 sectors with different scores) + distance (bullseye/triple/double rings)
- Human silhouette: every direction on the same elliptical ring scores the same, so no angle is needed — only the single distance dimension

3.3 Elliptical Distance Calculation (Calculate Score)

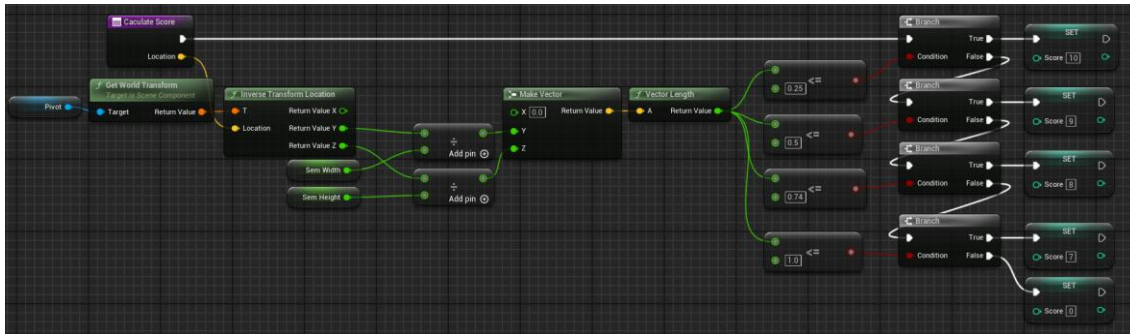
Purpose

Converts the hit point on the elliptical scoring rings into a normalized distance so the elliptical rings' score check can be done with simple distance thresholds just like a circular target.

Score zones

Distance condition	Score	Zone
Normalized distance ≤ 0.25	10	X ring (center)
$0.25 < \text{distance} \leq 0.5$	9	Inner ring
$0.5 < \text{distance} \leq 0.74$	8	Middle ring
$0.74 < \text{distance} \leq 1.0$	7	Outer ring
Distance > 1.0	0	Miss

Blueprint screenshots



Key points

- The distance thresholds were obtained by manually dragging an object to measure each ring's radius on the target face, the same method as the dartboard
- The cascading checks go from inside out: first 0.25 (10 points), then 0.5 (9 points) if not met, and so on; each hit point falls into exactly one zone
- The mathematical principle of elliptical normalization

4. Complete Shooting Flow

Execution flow

1. The player grabs the gun → caches the VRShoot component reference → determines the hand → switches to the shooting input mapping
2. The player holds the gun and pulls the trigger → BPC_04_VRShoot sets Trigger Click to true
3. The gun's Tick polls the three variables (not firing + trigger pressed + cooldown over) → conditions met → sets the states → calls Fire
4. Fire casts a ray from the muzzle → hits the target → Cast validation → calls the target's On Shoot(impact point)
5. The target moves the bullet hole → Calculate Score computes the score → Add Score awards points per the scoring mode
6. Fire checks the GameManager's attempts → if valid, Consume Attempt consumes one shooting opportunity
7. Fire plays the fire sound, slide recoil animation, and muzzle flash effect
8. After Delay(Interval Time), Can Fire? returns to true and the gun can fire again
9. The player releases the trigger → Is Fire? returns to false → pressing again is required for the next shot
10. The player puts down the gun → restores state variables and input mapping → clears references → Check Can End Game

5. Core Principle Explanation

5.1 Elliptical Normalization Principle

Core principles

Any point (X, Z) on an ellipse satisfies the ellipse equation: $(X \div \text{semi-width})^2 + (Z \div \text{semi-height})^2 = 1$. After dividing the hit point's X by the semi-width and Z by the semi-height, all points on the outermost elliptical ring have a normalized coordinate length of exactly 1 — the ellipse is "squashed" into a unit circle. All concentric elliptical rings become concentric circular rings, turning the elliptical problem of "distance to center varying with direction" into the simple ring check of "normalized distance versus fixed thresholds".

Geometric meaning

- A normalized distance of 1.0 is exactly the outermost elliptical ring's boundary; every ring's threshold is a ratio relative to the outermost ring
- X (the narrow axis) is divided by the smaller semi-width and Z (the tall axis) by the larger semi-height, separately compressing both directions onto the same scale
- Any hit point on the same elliptical ring (whether high, low, left, or right) has the same normalized distance, so the same ring scores the same, matching the silhouette target's scoring rules